

ALAMATIN

Panduan Dataset & Fine-tune NER (BERT)

Untuk: ML & Evaluation Lead
COMPFEST 18 AI Innovation Challenge

Disusun: 13 Agustus 2026
Repo: github.com/yongkytristan/ALAMATIN

1. Cara Pull / Clone Repo

Semua yang Anda butuhkan (dataset, kode, dokumentasi) ada di SATU repo public ini. Anda TIDAK perlu akses ke repo internal/private tim.

1.1 Kalau belum pernah clone

```
git clone https://github.com/yongkytristan/ALAMATIN.git
cd ALAMATIN
```

1.2 Kalau sudah punya clone-nya, tarik update terbaru

```
git checkout main
git pull origin main
```

Akses: Repo ini sudah punya 3 collaborator: yongkytristan, Kevinsweep, JasonEvan. Kalau akun GitHub Anda belum terdaftar, minta project owner invite dulu lewat Settings -> Collaborators di GitHub.

1.3 Verifikasi setup (opsional, tapi disarankan)

Repo ini tidak butuh dependency pihak ketiga untuk pengecekan dasarnya (stdlib Python saja). Jalankan ini untuk memastikan clone Anda sehat:

```
python scripts/check_repository.py
python -m unittest discover -s tests
```

Kalau semua lulus (OK), clone Anda sudah benar dan siap dipakai.

1.4 Alur kerja branch (wajib diikuti)

- Branch baru: feat/<nama-singkat>, contoh feat/ner-token-alignment atau feat/ner-finetune-v1 (lihat CONTRIBUTING.md).
- JANGAN commit langsung ke main.
- Pesan commit format: tipe: deskripsi, contoh "feat: add token alignment for subword labels".
- Buka Pull Request ke main, minimal 1 reviewer approve sebelum merge.
- JANGAN commit checkpoint model (.bin/.pt/.safetensors) ke Git -- sudah di-blok otomatis, limit file 10 MiB per scripts/check_repository.py.

2. Lokasi & Struktur Dataset

Dataset training NER sudah jadi dan sudah lolos audit kualitas. Lokasinya:

```
data/synthetic/
  train.json          <- 4.500 contoh, untuk training
  val.json            <- 750 contoh, untuk validasi selama training
  test.json           <- 750 contoh, test set sintetik
  generation-summary.json <- checksum, seed, distribusi noise per split
  audit-sample.json   <- 50 sampel yang sudah diaudit manual (referensi saja,
                        JANGAN dipakai untuk training/evaluasi model)
```

Penting: val.json dan test.json BUKAN nama file lama. Kalau Anda pernah lihat referensi ke dev.json atau test_synth.json di commit history lama, itu nama sebelum di-rename -- gunakan val.json/test.json yang sekarang.

2.1 Jumlah data per split

Split	Jumlah contoh	Base address unik	Kegunaan
train.json	4.500	1.500	Training model
val.json	750	250	Validasi / pilih checkpoint terbaik
test.json	750	250	Evaluasi akhir (sintetik)

Setiap "base address" dirender jadi 3 varian noise berbeda (variants_per_base=3). Satu base address dan SEMUA variannya dijamin hanya ada di satu split saja -- tidak ada kebocoran (leakage) data yang mirip antar train/val/test.

2.2 Kalau butuh volume lebih besar

1.500 base address untuk training itu skala awal (v1). Kalau butuh lebih banyak data untuk hasil fine-tune yang lebih baik, generate ulang dengan angka lebih besar (deterministik, hasilnya reproducible):

```
python scripts/generate_synthetic_addresses.py \
  --seed 20260813 \
  --train-bases 8000 --val-bases 1000 --test-bases 1000 \
  --variants-per-base 3 \
  --output-dir data/synthetic
```

Naikkan angka sesuai kebutuhan; skema label dan formatnya tidak berubah.

3. Dokumentasi Format & Isi Dataset

3.1 Struktur file JSON

Ketiga file (train/val/test.json) punya struktur identik:

```
{
  "schema_version": "1.0.0",
  "generator_version": "1.0.0",
  "template_version": "1.0.0",
  "label_order": ["O", "B-JALAN", "I-JALAN", ... ], // 21 label
  "examples": [
    {
      "id": "...", "categories": [...], "tokens": [...], "labels": [...]
    }
  ]
}
```

3.2 Contoh satu baris data asli

```
tokens: ["JL.", "AHMADY", "ANI", "NOMER", "177", "DESA", "CIBUNAR",
         "TAROGONG", "KIDUL", ",", "KAB.", "GARUT", "44129"]
labels: ["B-JALAN", "I-JALAN", "I-JALAN", "B-NOMOR", "I-NOMOR",
         "B-KELURAHAN", "I-KELURAHAN", "B-KECAMATAN", "I-KECAMATAN",
         "O", "B-KOTA_KABUPATEN", "I-KOTA_KABUPATEN", "B-KODEPOS"]
categories: ["abbreviation", "case_upper", "missing_provinsi",
            "missing_rt_rw", "separator", "typo"]
```

3.3 Skema label (WAJIB import, jangan tulis ulang)

10 entitas, BIO tagging, 21 label total. JANGAN hardcode label ID sendiri -- selalu import dari implementasi resmi:

```
import sys; sys.path.insert(0, 'src')
from alamin.label_schema import (
    ENTITY_TYPES, BIO_LABELS, LABEL_TO_ID, ID_TO_LABEL,
    SCHEMA_VERSION, validate_bio_sequence,
)
```

Entitas	Contoh
JALAN	Jl. Asia Afrika, Gg. Melati
NOMOR	No. 12, Nomor 8B
RT / RW	RT 04, RW 09
KELURAHAN	Kel. Braga, Desa Cibiru Wetan
KECAMATAN	Kec. Sumur Bandung, Kecamatan Coblong
KOTA_KABUPATEN	Kota Bandung, Kabupaten Bandung
PROVINSI	Jawa Barat, Jabar
KODEPOS	40111 (hanya angkanya)
DETAIL_LOKASI	Blok C2, Apartemen X Lt. 5, depan Masjid Al-Ikhlas

Catatan PII: Nama penerima dan nomor telepon BUKAN entity NER -- muncul sebagai token literal [NAME]/[PHONE] berlabel O. Jangan buat label baru untuk itu.

3.4 12 kategori noise yang ada di data

Kategori	Artinya
abbreviation	Bentuk singkatan (Jl./Jln/JL, Kel./Ds., dst.)
case_upper/lower/title	Variasi kapitalisasi seluruh teks
typo	Satu typo karakter (swap/hapus/dobel) per contoh
separator	Koma disisipkan sebagai token O di antar komponen
gang	Pakai "Gang/Gg." ganti penanda jalan biasa
missing_provinsi/kodepos/rt_rw	Field opsional itu dihilangkan
prefix_junk	Awalan seperti "Alamat:", "Kirim ke:"
instruksi	Instruksi kirim di akhir, contoh "kirim setelah jam 5 sore"
pii_mixed	Token [NAME]/[PHONE] ikut muncul, berlabel O
admin_conflict	Kode pos sengaja tidak cocok, tapi tetap berlabel KODEPOS (untuk uji Validator nanti, bukan untuk NER)

3.5 Jaminan kualitas data (sudah diverifikasi)

- Label 100% benar by construction -- dihasilkan langsung dari komponen penyusun, bukan dilabeli belakangan.
- Semua urutan BIO sudah divalidasi otomatis (validate_bio_sequence) saat generate -- tidak ada label cacat.
- Tidak ada kebocoran data antar split (anti-leakage, sudah diuji).
- Sudah diaudit manual 50 sampel lintas semua kategori noise -- lihat docs/synthetic_audit_log.md untuk detail temuan & perbaikannya.
- Tidak memakai data alamat pribadi/nyata -- hanya rantai wilayah resmi (provinsi/kota/kecamatan/desa/kode pos) + nama jalan & landmark sintetis.

3.6 Dokumen wajib dibaca sebelum mulai coding

File	Isinya
docs/label_schema.md	Kontrak label lengkap: definisi entitas, aturan boundary, contoh positif/negatif/ambigu
docs/synthetic_generator.md	Cara kerja generator, kategori noise, cara regenerate
docs/synthetic_audit_log.md	Hasil audit 50 sampel + bug yang sudah diperbaiki
docs/evaluation_protocol.md	Metrik wajib (F1 per-entitas, exact match, dll.) supaya hasil bisa dibandingkan ke baseline
src/alamat/label_schema.py	Implementasi resmi label -- import, jangan tulis ulang
src/alamat/evaluation_metrics.py	Implementasi resmi metrik evaluasi

4. Persyaratan Teknis NER & Fine-tune BERT

4.1 Environment

- Python 3.11+ (sama seperti requirement utama repo).
- GPU disarankan untuk fine-tune -- Google Colab (gratis) cukup untuk skala data v1 ini (indikasi dari rencana awal: 50k sampel x 3 epoch ~30-60 menit di GPU Colab).
- Inferensi akhir tetap harus bisa jalan di CPU (untuk Docker/demo), jadi catat juga latensi CPU saat evaluasi.

4.2 Dependency (belum ada -- ini bagian Anda)

requirements.lock sengaja masih kosong (repo ini baru pakai stdlib Python). Begitu Anda mulai pakai library ML, isi requirements.lock dengan versi exact + hash. Library yang kemungkinan dibutuhkan:

```
torch
transformers
datasets
segeval
accelerate # opsional
```

Model base: indobenchmark/indobert-base-p1 (utama, ~498 MB, lisensi MIT). Cadangan: indobenchmark/indobert-lite-base-p1 (~47 MB) kalau ukuran/kecepatan jadi masalah.

4.3 Urutan kerja WAJIB: token alignment dulu, baru fine-tune

Ini sumber bug paling umum di token classification. Sebelum fine-tune model:

- Tokenisasi pakai `is_split_into_words=True` (karena data sudah berupa token list, bukan raw string).
- Pakai `word_ids()` dari tokenizer untuk menyelaraskan label ke sub-token.
- Sub-token lanjutan (bukan token pertama dari satu word) diberi label -100 supaya tidak dihitung di loss.
- Tulis unit test untuk kasus subword, tanda baca, RT/RW, angka, dan whitespace SEBELUM training penuh (ini persis task ALM-017 di GitHub Issues).

4.4 Setelah fine-tune

- Pin base model revision + semua hyperparameter (lr, batch size, epoch, seed) -- harus bisa diulang dari command/config yang tercatat.
- Evaluasi pakai `src/alamat/evaluation_metrics.py`: F1 mikro, F1 per-entitas (fokus RT/RW/KELURAHAN), exact match per alamat, latensi CPU.
- JANGAN pakai kata "confidence" sebelum model score dikalibrasi -- sebut `model_score` dulu.
- JANGAN commit checkpoint model ke Git. Push ke HuggingFace Hub (public), backend nanti unduh otomatis saat container start.
- Data sintetik ini TIDAK BOLEH dicampur ke sealed real test (ALM-014/035) -- itu evaluasi satu-kali yang harus tetap independen.

5. Urutan Kerja & Referensi GitHub Issues

Issue	Judul	Status	Catatan
ALM-005	Skema label NER	Done	Sudah jadi dasar dataset ini
ALM-010	Generator alamat sintetik	Done	Dataset ini hasilnya
ALM-011	Audit 50 sampel sintetik	Done	Lihat synthetic_audit_log.md
ALM-015	Baseline regex/rule	Belum	Independen, boleh paralel
ALM-016	Baseline libpostal	Belum	Independen, prioritas lebih rendah
ALM-017	Token alignment pipeline	Belum	KERJAKAN INI DULU
ALM-018	Fine-tune & evaluasi NER v1	Belum	Setelah ALM-017 selesai

Urutan yang disarankan (mengingat deadline milestone M1):

- 1) ALM-017 -- token alignment (cepat, teknis, wajib benar dulu).
- 2) ALM-018 -- fine-tune BERT (bagian terbesar & paling menentukan; ini yang membuat proyek memenuhi syarat wajib lomba "model harus di-fine-tune").
- 3) ALM-015 (baseline regex) -- bisa diambil paralel, misalnya sambil training BERT berjalan di background/Colab.

Ada pertanyaan?

Hubungi Data & Research Lead (project owner) di tim, atau baca ulang dokumen di docs/ -- semuanya sudah disiapkan supaya Anda bisa mulai coding tanpa perlu menunggu konfirmasi lebih lanjut.